

Troubleshooting

Every entry below is symptom -> check -> fix, grounded in the code that ships in this repo, not in how a hook is supposed to behave. Where a command is shown as `<crew>/hooks/scripts/...`, that is `${CLAUDE_PLUGIN_ROOT}/hooks/scripts/...` inside a command file, or the plugin's cache directory when you run it by hand — see the shorthand in [Memory and Obsidian](#).

Stale Install

Symptom: a command from this guide (`/crew:status`), a new hook, a config key) does not exist, or behaves like an older release.

- **Check the installed version against the repo.**

```
cat ~/.claude/plugins/installed_plugins.json | grep -A3 '"crew"'
```

Compare the commit or version it records against `plugin/crew/.claude-plugin/plugin.json`'s version in the marketplace clone. `claude plugin update` decides whether to re-copy a plugin by comparing the **declared version**, not its content — a content change that ships with no version bump means every already-installed copy reports "already at the latest version" forever, holding the old code, with nothing in the repo itself looking wrong (`CLAUDE.md`, "Stop and ask"). This class of bug is why `/crew:status`'s absence, or a hook that clearly predates a documented fix, is worth checking here before assuming the fix is wrong.

A related, sharper case: on a `directory-source` install (a dev checkout registered as a local plugin path rather than pulled from a marketplace), `installed_plugins.json` can name a version string with **no relationship to the code that actually executes** — the tree runs whatever is on disk regardless of what that file says (`TOD0.md`, "`installed_plugins.json` records 0.19.56 while 0.19.63 executes"). On that install shape, trust the code over the version string.

- **Fix:**

```
claude plugin update crew@useful-claude-add-ons
```

then **restart** `claude` — a running session does not pick up a re-copied plugin. If the update reports "already at the latest version" and you can see the fix in the marketplace repo's `main`, the version was not bumped when it should have been; that is a defect to file against the plugin, not

something a client-side retry fixes.

- **Check it worked:**

```
/crew:status
```

Read-only, at most 40 lines, dispatches nothing. Its `config` line names which config file it read (`.crew/crew.json` for 1.0, `.crew/config.json` for 0.20); codemap reports behind: `<subsystem>` when a map's anchor and the path diff both say it is stale.

Hook Noise

Every hook crew registers, from `plugin/crew/hooks/hooks.json`, and the config key that turns its behaviour up or down. All events are registered once for bash and once for PowerShell (`shell: "powershell"`), so a Windows machine with both bash and pwsh on PATH fires both halves of every pair — that is what "even counts" in the file means, not a bug.

Event	Script	What it does	Switch
SessionStart	handoff-read.sh	resets its once-per-session markers; reads back <code>.work/HANDOFF.md</code> only when <code>memory.inject</code> is false	— (always runs)
SessionStart	platform-sync.sh	detects/repairs <code>platform.{os,wsl,shell,windows,HostIp}</code>	— (always runs)
SessionStart / UserPromptSubmit / PostToolUse / SubagentStart	crew-context.sh	injects code-map slices and vault recall, tracks the token budget	<code>context.enabled</code> (whole hook), <code>memory.inject</code> (codemap, handoff and vault injection — on by default since 1.0.0)

Event	Script	What it does	Switch
UserPromptSubmit	approval-hook.sh	records or refuses a /crew:approve <id> receipt	always records; enforcement depends on scope.mode
PreToolUse (Bash/PowerShell)	promote-gate.sh	the six command/production guards plus mergeGate	guards.terraformApply, guards.forcePush, guards.adminMerge, guards.mergeGate, guards.cloudDestructive, guards.sqlDestructive, guards.prodDatabase, guards.prodServer
PreToolUse (Write Edit)	role-write-guard.sh	refuses a write outside the dispatched role's declared scope	guards.roleWrites (block/report/off)
PreToolUse (Bash PowerShell)	cloud-guard.sh	destructive aws/az commands, wrong- identity commands, and dispatches of workflows listed in environments.workflow s (guards.deployWorkflow) — judged only when every word on the line is a plain literal or a single-quoted word; anything else asks, and is refused unattended	guards.cloudGuard (block/report/off)
PreToolUse (Bash PowerShell)	cloud-guard.sh, environment layer (T-0005)	a terraform apply judged by its target environment, and a destroy never applied	environments.nonProd (repo only; globs naming non- production workspaces), environments.prodUnattended

Event	Script	What it does	Switch
		unattended	(both layers; true only when both say true)
PreToolUse (Write Edit MultiEdit NotebookEdit Bash PowerShell)	scope-guard.sh	plan-approval + ticket scope guard	scope.mode (off/report/block/auto), scope.allowCliApproval
PreCompact	handoff-write.sh	writes the handoff note before compaction	context.autoWrapUp, context.handoffPath
Notification	notify.sh (crew_notify.py hook)	a Question / Needs permission ping when Claude stopped on a permission prompt, an AskUserQuestion or an elicitation; never idle_prompt; once per waiting episode	notify.provider (none or null disables it), notify.events, notify.questionTypes
Stop	verify-gate.sh	runs .crew/verify.json's checks	verifyGate (boolean)
Stop	context-watch.sh	nags for a handoff near the context budget, drives auto-clear	context.enabled, context.warnAt, context.budgetTokens, context.reserveTokens, context.autoClear.*
Stop	completion- audit.sh	diffs the whole tree against the ticket's scope base	scope.mode

- **Blocker pings (notify.events has blocker).** Stop gate refused means the verify gate or the completion audit refused the same ticket at Stop twice in a row (crew_notify.py stop

records it; the gates call it once their harness-only change lands). Lane stalled means /crew:autopilot found another runner's in-flight marker stale (a dead pid, or no heartbeat inside T-0049's 30-minute TTL) and names the owner's crew_inflight.py clear; Lane state unknown means it could not tell. Approval waiting is autopilot stopped at approve.

- **Silence one hook without touching the rest:** set its own key. guards.roleWrites: off, guards.cloudGuard: off and scope.mode: off are already the shipped defaults — a noisy session usually means one of these was turned on somewhere (repo or machine-global) and forgotten, not that the default changed.
- **Silence the SessionStart context specifically:** {"memory": {"inject": false}} stops the context hook; handoff-read.sh then prints the handoff on resume instead.
- **See what the context hook is actually doing:**

```
python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats
```

Reports emissions, injected characters, dedup hits, and vault-recall hit/miss/skip counts with a reason breakdown (see "Obsidian" below). The log itself is <git-common-dir>/crew/context-log.jsonl (or .work/crew/context-log.jsonl outside git), shared across every worktree of the repo. - **A hook fails and you cannot tell why:** every wrapper here resolves its own Python (python3/python/py) rather than assuming one is present, and every one that can block fails **closed** — never silently — when no interpreter resolves. See "no-python fail-closed" under Scope, below; the same shape applies to scope-guard.sh, role-write-guard.sh, cloud-guard.sh and verify-gate.sh.

Review Loops

Each ticket gets **two** review rounds, reserved before the reviewer runs, tracked in <git-common-dir>/crew/review/<ticket>.json — shared by every worktree of the repo, so a second worktree of the same repo spends the same budget (review_ledger.py).

- **Symptom: review keeps re-running / "budget exhausted". Check:**

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --status
```

A round's outcome is CLEAN, FINDINGS or INCOMPLETE. A completed round 2 — either FINDINGS or INCOMPLETE — leaves the ticket state REVIEWED, so its FINDINGS can still be accepted. A **third**

reservation attempt is refused outright and the state becomes NEEDS_REPLAN; that refusal, an explicit --reject, and --reject --supersede-accepted on an ACCEPTED ticket are the only ways into NEEDS_REPLAN (autopilot's crew_autopilot.py auto-reject, under autopilot.maxAutoReplans, is a --reject by the name autopilot (policy: autopilot.maxAutoReplans)). **Fix:** a final round with 0 BLOCK from a Codex or Kimi reviewer closes itself: review: auto-accept: eligible, then --auto-accept --follow-up <id> writes an auto-accepted receipt and its FIX/NIT lines go verbatim into one follow-up ticket. A review: auto-accept: refused - <reason> line names what stopped it (any BLOCK, INCOMPLETE, not the final round, an open healer skip, a count it could not read, a same-family Claude-fallback round, a provider or family it could not tell, or a verdict recovered from stray lines - ignored_lines above 0, or missing or unreadable, which is could-not-tell, as on a round recorded before L-0576 wrote the count). Otherwise own the FINDINGS with --accept --by <who> (only the most recent completed round, only once, never once NEEDS_REPLAN; a name starting auto: is refused), or write a new plan and get it approved — crew_ticket.py approve on a NEEDS_REPLAN ticket opens a fresh budget of two rounds counted from the successor plan; the rounds already spent stay in the ledger and are not erased. With notify on, a spent budget whose last round carries a BLOCK sends the Review out of rounds ping when /crew:autopilot stops on it.

- **Symptom: a round came back INCOMPLETE. Check:** the review: lines, or failure_class in .work/tickets/<id>/review.json. An INCOMPLETE round has one of three classes. tool means the answer never arrived intact: a timeout, a bad or unknown exit, empty output, or a failed Codex stream. tree means a bundle part or web-test report changed under the reviewer. reviewer means the output arrived and broke the contract. Harmless prose, a heading or a code fence beside well-formed findings, with every part acknowledged and exit 0, no longer does that: the round is FINDINGS, and the ignored lines are printed on a review: FINDINGS kept; ... line and kept in review.json's ignored_text (ignored_lines is their count). A stray line beside CLEAN, a misformatted contract line (-FIX|..., fix|..., a | table row) or a line matching the shortfall wording list ("skipped", "truncated", "could not review") is still INCOMPLETE reviewer. That list cannot catch every admission: one it misses ("I only inspected one of the nine files") is ignored as prose and the round stays FINDINGS, so read the ignored lines. **Fix:** a tool round is refunded automatically, up to two per plan. The line reads review: round N was a tool failure (...); refunded. review_run.py retries it once by itself (L-0514), except a usage limit or a timeout; review: retry: not retried - <why> says when it did not. Only the failed round is given back: the rerun /crew:review reserves a new round, charged like any other

unless it is a tool failure too, so a ticket with one charged round that reruns and gets FINDINGS has spent the budget. If Codex is out of quota, use the next eligible provider. A third tool failure under one plan reads NOT refunded - refund limit 2 per plan reached and counts. A reviewer or tree round always counts. Rebuild the bundle (tree), or rerun and read what the reviewer wrote (reviewer).

- **Symptom:** `/crew:done` refuses with a receipt error. **Check:**

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --check-receipt
```

A CLEAN verdict writes a receipt automatically; a FINDINGS verdict only becomes one through `--accept`, or `--auto-accept` on a final 0-BLOCK round. An auto-accepted receipt stands only while its round still reads 0 BLOCK with the same lines from the same Codex or Kimi reviewer the receipt names. `--check-follow-up` fails until the follow-up's `direction.md` quotes every line verbatim (a line the receipt carries twice, twice), and on a non-UTF-8 file or an unknown receipt kind. `/crew:done` does not run it yet (L-0568 adds it to check 1), so run it yourself before closing. `--check-receipt` rebuilds the review bundle from the receipt's recorded base and fails unless the hash still matches, the receipt is for the **latest** recorded round, and the state is not NEEDS_REPLAN — so editing a file after the reviewer read it, or after the receipt was written, invalidates the receipt even though nothing about the ledger itself looks wrong. One exception, the delta gate: after a catch-up merge, a version bump or an anchor-only refresh, committed on a clean checkout, the receipt is kept (receipt kept by delta gate: ...) when the ticket's own delta is byte-identical to the reviewed one. Its stale line names what differed (delta gate: <path> ..., excluded path changed, not clean, no train entry binds the integration ref - the merge train is not armed in this clone, so the gate keeps nothing yet); a code map, rules file or diagram may move only its anchor sha after review. **Fix:** if the edit was deliberate, get the ticket reviewed again (spends the next round); if it was accidental, revert the edit and re-check. crew's own bookkeeping written after acceptance (the verify gate's records, a metrics row, the scope base) never stales a receipt: the bundle leaves `crew_ticket.CREW_BOOKKEEPING_PATHS` out, whatever `.gitignore` says.

- **Symptom:** a receipt accepted under an older crew reads stale after the upgrade. The upgrade to the release that brought T-0068 drops crew's bookkeeping from the bundle, so a receipt whose bundle held a non-ignored bookkeeping file (a repository that does not ignore `.crew/*`) no longer matches the rebuilt hash. **Fix:** one re-review of that ticket. Later bookkeeping writes cannot stale the new receipt.

- **Symptom: Codex hit a usage limit.** The probe printed PROBE=limited (exit 5) with the error on PROBE_DETAIL=..., or a round printed review: codex usage limit in round N: **Cause:** Codex's usage limit, rate limit, quota or spend cap - its own message is quoted. **What happens:** that round runs on the Claude reviewer, announced as same-family (codex limit) and not independent; the limit is recorded in <git-common-dir>/crew/review-limit/<ticket>.json and applies to the next round only. The round is spent like any other. To check Codex yourself without spending one:

```
python3 "<crew>/hooks/scripts/review_run.py" --root . --ticket <id> --scratch <dir> \
--provider codex --probe
```

- **Symptom: you want to send a ticket back without spending the third round.**

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --reject --by <who>
```

Refuses on a ticket already ACCEPTED or already NEEDS_REPLAN. An ACCEPTED ticket whose head proved unshippable takes the explicit flag, the owner's call:

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --reject --by <who> \
--supersede-accepted
```

It moves the ticket to NEEDS_REPLAN, keeps the old receipt in the superseded list with who and when, and clears it; only an approved successor plan continues. It refuses, changing nothing, on any other state, a name starting auto:, or a receipt or round it cannot read. --by is a recorded name, not a check of who is calling.

- **Symptom: the receipt names the wrong acceptor** (a peer ran --accept --by under its own name for a decision the owner made).

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --correct-acceptance \
--by <who> --reason "<why, one line>"
```

Rewrites accepted_by on an owner-accepted receipt and appends {round, was, now, reason,

at} to acceptance_corrections (shown by --status). Nothing else changes, so --check-receipt answers the same. Refuses a clean or auto-accepted receipt, a name starting auto:, the name already recorded, and an empty or multi-line --by or --reason. A wrong correction is fixed by another one; rows are never removed.

- **Symptom: crew_train.py acquire exits 1, or /crew:review stops with exit 10 and review-run: train: waiting behind <ticket>.** The clone's merge train is armed (L-0520; the gate round takes it since L-0526, and no round was spent) and an overlapping ticket holds it, or queued first on the same base. **Check:**

```
python3 "<crew>/hooks/scripts/crew_train.py" --root . status
```

Each waiting entry lists the ticket it is behind and every colliding pair (<mine> x <theirs>); touch: undeclared: <why> means that ticket's spec has no usable ## Touch, which overlaps everything. **Fix:** wait for the holder to land and release, then review again; fix an undeclared Touch in the spec. merge <base> first means the base moved in this ticket's Touch: run crew_train.py catch-up --ticket <id> (resolve any conflict), bump the version one past the base's, refresh the artifacts, commit, gate the merged head, and review it again if review_ledger.py --check-receipt reads stale: the delta gate keeps the receipt only when none of the ticket's own code moved and the clone's merge train is armed (it keeps nothing until crew_train.py arm). Then acquire again. could not tell (exit 3) means the train state could not be read — the message names the file; nothing is guessed.

- **Symptom: a lane holds the train and its session died.** status prints stale?: beside it (worktree missing, head already in the base, held for hours). Nothing releases it automatically, by design. **Fix:** once you are sure it is dead:

```
python3 "<crew>/hooks/scripts/crew_train.py" --root . release --ticket <id> --force \
--by <who> --reason "<why>"
```

The release is logged as a force-release event that the other lanes see.

- **Symptom: check-land says the base moved in Touch paths.** Another ticket landed changes to paths this ticket touches after it was gated, so the verdict covers a different tree. **Fix:** crew_train.py catch-up --ticket <id> (a merge; conflicts and rerere-replayed files are listed and left unstaged for you to inspect, git add and commit; a version file is never replayed and comes back conflicted), then bump the version one past the base's,

refresh the artifacts, commit, gate the merged head, review it again (`/crew:review`) if `review_ledger.py --check-receipt` reads stale, then `check-land` again, so the tree the gate passed is the tree that lands. `merge-tree: HEAD conflicts with <base>` is the same fix with a conflict to resolve first.

Scope: Approval and the Completion Audit

A ticket's contract lives in `.work/tickets/<id>/: spec.md`'s `## Touch` section names every path the ticket may change; `plan.md`'s `Files:` lines must each fall inside `Touch`. Two hooks hold a session to that contract — see [Daily workflow: scope and approval](#) for the contract itself. This section is what goes wrong with the approval and the audit.

- **Symptom: a done ticket "has no .work/tickets/ folder". Check:**
`.work/tickets/Complete/<id>/: crew_tracker.py archive --ticket <id>` moves a done or merged ticket's folder there (and, under Obsidian, its note to `<boardDir>/Complete/`), and `/crew:status` counts those apart (`<m> archived in Complete/`). **Fix:** none needed: every crew reader finds it there. To reopen it, move the folder and the note back by hand; `move --reopen` refuses an archived ticket.
- **Symptom: could not tell where <id> lives. Check:** both `.work/tickets/<id>/` and `.work/tickets/Complete/<id>/` exist (a half-finished archive, or a copy made by hand), or one of them cannot be read. **Fix:** compare the two, keep the right one, remove the stray copy. Crew never picks one.
- **Symptom: a lane worktree does not see my settings** (a CLI approval refused, `scope.mode` read as `off`, guards at their defaults). `.crew/*` is gitignored, so `git worktree add` makes a checkout with no crew config. **Check:** `/crew:status` prints `config inherited from the main checkout (<path>)` ... when the worktree reads the main checkout's `.crew/config.json` (crew 1.0.69+, T-0088), and `crew_config.py --root . --explain` starts with `repo layer: <path> (<source>)`. **Cause and fix:** a worktree with its own `.crew/config.json` or `.crew/crew.json` reads only those, never merged with the main checkout's - delete them to inherit. `/crew:status` names that case: ... the main checkout's `(<path>)` is not read A lane made before 1.0.69 almost always has one, a default that crew's `SessionStart` heal wrote there. (unknown) means git could not name the main checkout; then no default is written either. Of the shell and PowerShell readers (T-0096), the incident stand-down read (`_common.sh`, `promote-gate.ps1`), both cloud-guard no-python fallbacks and `auto-clear.ps1` inherit too; in the cloud guard's

fallback unknown counts as armed. So do the session hooks (notify, the handoff scripts, context-watch; crew 1.0.343, L-0680): a lane notifies with the main checkout's settings, and a relative handoff path still names a file in the lane. Since L-0681 the verify gate (both flavours), review_gate.py and the scope and completion wrappers inherit as well: an inherited "verifyGate": false stands a lane's Stop gate down, and with no python a lane whose main checkout has a config blocks writes and the Stop (the PowerShell 7 wrappers still allow one that strictly says scope.mode: off).

- **Symptom: which tickets still need my approval? Check:** /crew:status --approvals. It prints one ready-to-paste /crew:approve <id> line, with a why: <why> line under it, per open ticket whose approval is missing, stale or unaccepted, and nothing for merged, current or spec-only tickets (nothing needs approval when there are none). Approving a ticket it does not list changes nothing.
- **Symptom: an edit inside Touch is still refused. Check:** approval status.

```
python3 "<crew>/hooks/scripts/crew_ticket.py" status --ticket <id>
```

Returns approved (both spec.md and plan.md hash to what was approved), stale (either file changed since), or none. **Editing spec.md or plan.md after approval makes the approval stale immediately** — that is deliberate: an edited spec cannot silently widen what the guard accepts. Only a receipt from the user's own prompt counts by default; a cli-sourced receipt (written by crew_ticket.py approve directly, for tests/CI) is accepted only when .crew/config.json sets scope.allowCliApproval: true, and an autopilot receipt (from crew_autopilot.py approve) only while that is true and autopilot.approval still allows it. That includes a receipt written inside an autopilot.sleep.schedule window under autopilot.sleep.approval: once the window ends, the day value decides, and it may not allow it. The same holds for a manual /crew:autopilot sleep once its until passes. If settings prints sleep=unknown source=manual, <git-common-dir>/crew/autopilot-sleep.json could not be trusted (the warning names why): the stricter value applies per key until /crew:autopilot wake removes or replaces it. **Fix:** re-approve. The user types /crew:approve <id> again — the only other route is /crew:autopilot under an opted-in autopilot.approval; scope_guard.py refuses a Write/Edit under <git-common-dir>/crew/ in every mode but off, so a session cannot forge or refresh its own approval.

- **Symptom: you need to touch one more path mid-ticket. Fix:** amend spec.md's ## Touch (widen it), then approve again. There is no partial-approve; amending scope is edit-then-approve, same as any other spec change.

- **Symptom: the Stop prints outside-scope: (could not tell - ...).** The verify gate's scope line (`scope_report.py`) reads the active ticket's approved `.work/tickets/<id>/spec.md ## Touch`, as the completion audit does. The reason names the gap: no readable `spec.md`, no `## Touch` paths, a `Touch` that is not approved (approve it, or approve it again after a spec edit), a broken active-ticket pointer, or a pre-1.0 `.work/tickets/<id>.md` ticket (run `/crew:migrate`). Before L-0711 the line said every 1.0 ticket's file was missing. It is report-only: the refusal is `/crew:done check 3`.
- **Symptom: `/crew:done` refuses on "out of scope" for a file the edit guard never saw.** This is the Stop-time completion audit (`completion_audit.py`), not the edit guard. The edit guard only sees `Write/Edit/MultiEdit/NotebookEdit` tool calls; a `sed -i`, a redirect, a formatter or a `git mv` never reaches it. The audit instead diffs the **whole working tree** against the ticket's scope base (`scope_base.resolve` — the commit the ticket started from) across committed, staged, unstaged and untracked changes, so a shell-made write is caught here even though nothing blocked it at the time. A file byte-identical to main as last merged is not counted; a merged main: `could not tell` line (a detached HEAD, none of `origin/HEAD`, `origin/main` and main naming a commit, or a git error) means every merged-in file was counted, so check out the ticket branch and rerun. A missing `origin/main` alone is not that: a local main is used instead. An untracked merged-in file (after `git rm --cached`) is main's only when `git add` would record it identically: with `core.fileMode=false` its execute bit is ignored, as git ignores it. On the review side, `diffed-from-merged=could-not-tell` (or `fork: could not tell from --check-receipt`) means `git merge-base <start> <merged>` gave no answer, so a file main also changed shows main's lines as the ticket's: fetch, check the start commit still exists, and rebuild.
Check it directly, without waiting for a Stop:

```
python3 "<crew>/hooks/scripts/completion_audit.py" --check --ticket <id>
```

Fix: either the path genuinely needs to be in scope (widen `Touch` and re-approve), or revert the out-of-scope change. Files git ignores (including everything under `.crew/`) and `.work/` itself are outside what the audit can see at all — that is by design, not a gap to work around.

- **Symptom: the scope base is the merge-base with main on a repo whose branches come from development.** `scope_base.py --record` says "(fallback) recorded ... the merge-base with `origin/main`", and the review bundle or the completion audit lists hundreds of files the ticket never touched. With no `tickets.baseBranch`, the base branch is `origin/HEAD`'s target, then `origin/main`, then `main`, so a branch cut from development is

measured against `main` and the whole integration branch looks like this ticket's change.

Fix: set `"tickets": {"baseBranch": "development"}` in the repo's `.crew/config.json`, then `scope_base.py --root . --record <id>`. A fallback recorded against the old branch is re-derived ("re-derived ... was a merge-base guess against origin/main"); an exact record is never moved. A value that names no commit here, or a config that does not parse, reads as **could not tell**: `--record` exits 1, `--base` exits 3 with nothing on stdout, and the audit fails. It never falls back to `origin/HEAD` silently. Fix the value; do not unset it to make the error go away.

- **Symptom: the completion audit or the verify gate lists `.crew/.scope-base`, `.crew/metrics.md` or `.crew/.verify-gate.record.json`.** Only a crew from before T-0068 does that, in a repository whose `.gitignore` does not ignore `.crew/*`. **Fix:** update crew. These are crew's own bookkeeping (`crew_ticket.CREW_BOOKKEEPING_PATHS`), never a changed path for the audit, the gate or the review bundle; do not add them to Touch. A `.crew/` path the audit still lists (`.crew/verify.json`, `.crew/config.json`, a committed `.crew/incident.json` or `.crew/tfplan/` file, a session marker) is a real change: only the ticket-flow bookkeeping and the hook logs (`.crew/guard.log`, `.crew/.autoclear.log`) are left out, never a file crew reads as a trust input.
- **scope.mode values, and what "auto" means:** `off` (hooks do nothing, the default), `report` (allows everything, logs the row to `.crew/guard.log`), `block` (refuses out-of-scope writes and fails the completion audit), `auto` (report for the first ten tickets approved in this repo, then `block`). A config file that exists but fails to parse, or names an unknown mode, resolves to `block` and says so — the unknown never collapses into the permissive value.
- **No-python fail-closed, and why it is not a bug:** if none of `python3/python/py` resolves to a *proved* working interpreter, `scope-guard.sh` (and `completion-audit.sh`, byte-for-byte the same logic) does **not** silently allow the write. It checks whether `.crew/config.json` **provably** sets `scope.mode` to `off` — a crude, non-JSON textual check, because it runs in exactly the situation where python cannot parse the file for it — and only then exits 0. Anything it cannot prove `off` (a corrupt file, `report`, `auto`, `block`, or no `scope` key at all) fails **closed** with exit 2: "no usable python ... failing closed". Fix by installing a real Python 3, not by reading the closed refusal as a false positive.
- **"the python probe timed out" (the PowerShell scope guard, completion audit, approval hook and verify gate, crew 1.1.4, L-0690).** The probe gives each candidate 3 s and the whole walk 8 s; when it runs out of time, or kills a candidate at its bound, it says "the python probe timed out" (could not tell) instead of "no usable python", with the

same exit code. Whenever the probe finds nothing, `python probe:` lines follow on stderr: a summary (found, not-found, rejected or timed-out, the elapsed milliseconds and both bounds), then each candidate tried with its path, its time and what happened (killed-at-bound, exit-nonzero, not-python-proof, not-tried-budget-spent, ...). A timeout under heavy load is the machine, not a missing python: re-run when it is idle. The other PowerShell hooks and the bash twins do not report this yet.

Verify Gate Says **COULD NOT TELL**

`verify-gate.sh / .ps1 (T-0082)`. A rule counts as passed only when its wrapper ended with status 0 **and** left a completion record saying the rule exited 0. Anything else that is not a plain exit status is FAILED as "could not tell": the gate prints `VERIFY FAILED: <cmd>`, then `verify-gate: COULD NOT TELL (<reason>): <cmd>`, exits 2, and neither marker advances. The summary adds `verify-gate: N rule command(s) COULD NOT BE JUDGED - counted as FAILED`. The reasons:

- **"exit status N, above 128 and not a signal number - the rule's own status"** (N from 193 to 255; signals stop at 64): the rule itself exited that code - 255 is common from `ssh` or a shell error. It is still could-not-tell, because nothing distinguishes it from a crash code.
- **"exit status N: ended by signal N-128, or the rule's own status"** (N from 129 to 192): the rule was killed (137 is KILL, 143 is TERM) - an OOM kill, an outside timeout -s KILL, a CI step limit - or it really exited that code. Re-run the command by hand and see which.
- **"the rule's runner ended with status N before it recorded a result"**: the wrapper around the rule was itself killed before it could write the record, so the rule's own result is unknown.
- **"no completion record"**: the wrapper ended 0 but the record is missing, empty, not 1-3 digits or above 255 - the shape a native Windows kill that reports exit 0 leaves. Nothing about the rule can be told from that run.
- **"the rule's shell could not be started"** (`.ps1` only): the resolved bash vanished or would not launch for this rule. Before T-0082 this rule silently inherited the previous rule's pass.
- **"cannot create an output-capture file or a completion-record file" or "no usable bash resolved"**: the existing refusals, now named as could-not-tell.
- **"the gate received TERM while this command was running"** (`.sh` only): the gate itself was signalled mid-rule; it still exits 128+N. A killed `.ps1` gate prints nothing (PowerShell

has no signal trap); the marker not advancing covers it.

The command log carries status unknown for such a rule; the record sync leaves that rule's entry as it was and the tree-pass cache never stores it. The gate still waits as long as a rule runs - a hung rule is not ended by the gate.

The CI receipt (`ci_receipt.py`, L-0673) lists such a command as UNKNOWN, and so a command the gate named failed, skipped or could-not-tell but never finished (a gate killed mid-rule). A log whose `verify-gate: <N>s total` across line does not come after its last per-rule line records `log_complete: false`; when the gate also exited non-zero, the job summary says the list is partial. The list is informative only: what the receipt accepts did not change.

A Diagram Check Fails in CI with No Reason, or "Running as Root Without --No-Sandbox"

Headless Chromium, which `mmdc` drives, refuses to start as root unless puppeteer passes `--no-sandbox`, and CI containers run as root. A hand-written `_verify` case that calls `mmdc` bare fails there, and an older `_verify/run-all.sh` or `_verify/smoke.sh` threw the check's output away, so all the log said was FAIL `<name>`.

- **Use the ready case.** Copy `plugin/crew/skills/crew-setup/templates/cases/diagrams-render.sh` into `_verify/cases/` (crew-setup does this itself in a repo that has `.mmd` files). It passes the same `--no-sandbox` puppeteer config as `plugin/crew/skills/crew-diagrams/scripts/render.sh` every time, renders to a temp directory, prints `mmdc`'s own last lines under a failed source's FAIL line, and exits 77 (SKIP) where `mmdc` is not installed. `DIAGRAMS_DIR` points it at a directory other than `docs/diagrams`.
- **See why a check failed.** The current template runners print the last 5 lines of a failing check's output under its FAIL line and report exit 77 as SKIP, not a failure; a run with no failure but such a skip exits 77 itself, so crew's verify gate records it skipped, not verified. A repo set up earlier keeps its old runners: replace the counter line, the `check()` / `run()` function with the `CUR_OUT` / `cleanup` / `trap` lines above it, and the final count and exit lines from the templates. Copying `check()` alone breaks the old `_verify/smoke.sh` on the first exit 77 (SKIP is unset under `set -u`).

Autopilot in a Worktree, and the Refresh Check

- **Symptom: autopilot stops with cannot tell whether <id>'s direction is approved in a worktree.** `crew_autopilot.py` next looks for the ticket's `.work/INDEX.md` row in the checkout it runs in, and — since crew 1.0.349 — in the main checkout (the first record of `git worktree list --porcelain`) when this one has none. `.work/` is git-ignored, so a lane worktree made from another branch starts with no INDEX at all. **Check:** the stop's reason names every INDEX.md it asked, or why the main checkout could not be read (a failed listing, a bare repository). `--json` names the file that answered as `index_source`:

```
python3 "<crew>/hooks/scripts/crew_autopilot.py" next --root . --ticket <id> --json
```

Fix: add `<id> | ready | <risk> | <repo> | <title>` to either INDEX once the direction is agreed. Two other stops come from the same read: `index-disagreement` (both INDEX files have a row and the status cells differ — make them agree), and `folder-elsewhere` (the ticket's folder is only in the main checkout). Copy the folder with the `cp -r` the reason prints; autopilot never reads a spec or plan from another checkout, because the scope guard reads Touch from this one.

- **Symptom: the refresh check says fresh-uncommitted.** Every artifact the ticket reaches is current, but a refreshed file is not committed — the map, a diagram source or render, anything under the graph dir, or any other path under the refresh-artifact dirs (`.claude/rules/` included). `fresh` means current **and** committed, so `/crew:done` check 4 refuses it. **Check:** the `uncommitted:` line lists the paths:

```
python3 "<crew>/hooks/scripts/crew_refresh_check.py" --root . --ticket <id>
```

Fix: commit exactly those paths and re-run until it says fresh; autopilot does it for you as its `commit-refresh` phase. The commit changes no byte of the working state the review bundle is built from, so a review receipt stays current and no new round is needed. A file `graphify` writes and git ignores (`manifest.json`, `cache/`) never counts.

Wave Refuses a Cross-Session Dependency

A wave ticket may depend on a ticket another session works, written `<channel>:<id>` or `<channel>:<repo>:<id>` in the set file's `deps` or the INDEX row's (`depends on ...`). The wave

fetches `crew-coord/<channel>` from `coord.remote` (default origin) and counts the dependency closed only when exactly one claim for that id reads done. Every line about the peer's claim is peer-written data and ends `[peer-written]`.

crew_wave.py plan says	Means	Do
dependency <channel>:<id> is not closed: the peer claim reads working (...)	the peer is still on it; owner unknown in the brackets means its heartbeat is past the TTL	wait for the peer's done, or take the ticket out of this wave
... reads released (...); only done closes it	the peer gave the ticket back unfinished	agree with the peer who finishes it; released never closes it
dependency <channel>:<id> unknown: no claim for <id> on crew-coord/<channel>	nobody on the channel has claimed that id	check the id and channel with <code>crew_coord.py status --channel <channel></code>
... unknown: 2 repositories on crew-coord/<channel> hold <id>; name one as <channel>:<repo>:<id>	the short form is ambiguous	write the long form, with the repo key status prints
... unknown: its claim ... is corrupt (...)	the claim file does not parse	the claim's holder or the owner repairs it; the wave never guesses
... unknown: crew-coord/<channel> does not exist on <remote> or could not fetch ...	the channel is absent, or the remote cannot be reached	check the channel name and <code>coord.remote</code> , then plan again
... unknown: '<remote>' (<code>coord.remote</code>) is not a configured remote	<code>coord.remote</code> names no remote of this checkout	<code>git remote add <remote> <url></code> , or fix <code>coord.remote</code>
dependencies unknown: ...	a dependency is not <id>, <channel>:<id> or <channel>:<repo>:<id>	fix the set file's deps or the INDEX row; <code>crew_wave.py</code> set refuses a malformed one

Wave Refuses a Ticket's Contract

A ticket that built against a contract version (`crew_contract.py build-against`) keeps a binding in `.work/tickets/<id>/contracts.json`, naming the git remote it was built on. The wave checks each binding, on that remote, after the ticket's dependencies; run the same check alone with `crew_contract.py verify --ticket <id>` (exit 0 current, 1 changed, 3 cannot tell).

crew_wave.py
plan says

Means

Do

`contract <n>`
`v<N> changed`
`since <id> built`
`against it (...)`

the channel no longer
shows that version with the
bound hash, a matching
body, status built-against
and this ticket in built_by:
someone rewrote it

do not rebind: agree a new version (put `--name <n> --new-version --ticket <new id>`) and a new ticket on each side, then build against it

`contract <n>`
`v<N> unknown`
`(...)`

the fetch failed, the
binding's remote is no
longer configured, the
channel or the version's files
are missing, or the record is
corrupt

fix what the brackets name (remote, channel name), then plan again; nothing is assumed current

`contract`
`bindings unknown`
`(...)`

`contracts.json` cannot be
checked, does not parse or
holds no binding

restore the original file (its permissions, or a copy) if you have one; otherwise treat the contract as changed and agree a new version and a new ticket on each side. Do not delete it and run `build-against` again: that records whatever the channel holds now, a rewrite included. It is never read as "no bindings"

A newer version on the channel is information only and never refuses. The check writes nothing.

Promote Gate Blocks a Worktree Deploy

`promote-gate.sh / .ps1`, the `PreToolUse` hook on declared deploy commands. Since T-0505 it

judges **the tree the deploy runs from**: the Bash call's cwd, moved by a leading `cd <dir> &&` and named by any `git -C <dir>`. That tree must be a worktree of the same repository, clean, and at every literal sha the command names. `.crew/verify.json`, `.work/PROMOTIONS.md` and the `.crew/.approved-<env>-<sha>` markers are still read from the session's project directory.

- **"the tree this deploy runs from ('...') is dirty"** names the tree it judged. If that is the main checkout while you meant a worktree, run the command there: `cd <worktree> && <deploy>`, or enter the worktree first. A dirty main checkout no longer blocks a clean worktree.
- **"has rows for sha X ..., but the newest row is not all-pass"** (L-0665): the newest `.work/PROMOTIONS.md` row for that environment and sha is a failure, a not-run row from `crew_ghdeploy.py` record, or anything but three pass cells; an older pass no longer counts. Re-run `/crew:promote <upstream>` for that sha and let it append its row.
- **"DEPLOY NOT RECORDED" for a sha no deploy ran at**: before L-0689 a command that was a FRAGMENT of a declared deploy (`git rev-parse HEAD, development`) matched it, and the gate wrote `.crew/.deploy-in-flight` for a deploy that never ran. A command now matches only when it contains the declared text. A stale marker left by an older crew names that sha; check that no deploy ran, then delete `.crew/.deploy-in-flight` by hand. If a real deploy is now unmatched, its runs are shorter than the declared text: declare the shortest text every real run contains.
- **"no all-pass row for sha X"** where X is the worktree's sha: the upstream environment passed a different sha. Promote the worktree's sha upstream first; a row for the main checkout's sha does not carry over.
- **"PROMOTION BLOCKED (workflow dispatch): python could not be found ..."** (L-0664, PowerShell tool): the map declares a dispatch deploy, the command names gh with workflow or dispatches, and no python resolved to read it. Install Python 3 or put it on PATH; this is not a pass.
- **"PROMOTION BLOCKED (workflow dispatch): the gate could not tell ..."** (T-0062 on the Bash tool, L-0664 on the PowerShell tool): the command is a `gh workflow run` or `gh api .../dispatches` and the map declares a dispatch deploy, but crew cannot read the line with certainty (a variable, a substitution, double quotes, a pipe, `--json`, `--input`, `-F k=@file`, a workflow id or display name), or a declared workflow fits no single environment. Spell it as the declared deploy is - plain literal words, the workflow's file name, the declared inputs with their declared values.
- **"the command names commit '...', but the tree it runs from ... is at ..."**: a literal

ref=<sha> that is not the tree's HEAD. Deploy from a tree at that sha, or drop the literal.

- **"changes directory after it starts"** (a later `cd`, `bash -c 'cd ...'`, `env -C`, `make -C`), **"names more than one tree"**, **"not a form the gate reads with certainty"** or `--git-dir`: the gate will not guess. Put a single literal `cd <dir> &&` first, or use `git -C <dir> -` and note the directory the command itself runs in must be clean too.
- **"index entries flagged skip-worktree or assume-unchanged"**: those flags hide edits from `git status`; clear them in the tree being deployed.
- **"has uncommitted changes: it ..."** also fires when the map is deleted or untracked, or edited under `skip-worktree`: the gate compares the file with HEAD's copy, not with `git status`.
- **".crew/verify.json in the project dir ... has uncommitted changes"**: commit or revert the map; an uncommitted map is not policy.
- Never route around a block by running the deploy yourself with `!. /crew:promote` fixes the precondition the message names and asks you only for a `requireHuman` yes or a genuinely interactive step.

The Run Could Not Be Identified

`crew_ghdeploy.py identify` (a github environment under `/crew:promote`, L-0645) ends with `result=could-not-tell reason=<code>` and exit 3. It names a run only when exactly one new run fits; it never picks among candidates.

- **two-candidates**: two new `workflow_dispatch` runs of that workflow, on that ref, by the same actor, appeared in the window - usually two dispatches at once. Set `correlationInput` on the entry and put that input in the workflow's run-name, so each run's title carries its id.
- **none-in-timeout**: no new run within `identifySeconds` (default 120). Check the dispatch really ran (its own Bash call, not blocked), then raise `identifySeconds` (10 to 900) if runs are slow to appear.
- **correlation-not-found**: new runs appeared but none had the correlation id in its title. The workflow's run-name must include the `correlationInput` input; there is no fallback to time.
- **stale-prepare**: prepare ran more than 600 seconds ago. Run prepare again, then dispatch.

- **state-file-missing / state-file-unreadable:** run prepare first; never edit `.crew/.ghdeploy/<env>--<N>.json` by hand.

Do not pick the run from the Actions tab and carry on: run `crew_ghdeploy.py record`, which writes `could-not-tell` and a `not-run` row, and decide from there.

Cloud Guard False Positives

`cloud-guard.sh / cloud_guard.py`, a PreToolUse hook on Bash and PowerShell. **Ships off** (`guards.cloudGuard: off`); turn it on with `{"guards": {"cloudGuard": "report"}}` first to see what it *would* refuse before enforcing with `"block"`.

- **SQL text that merely looks destructive is not blocked.** `DROP/TRUNCATE` inside a string literal or a comment does not count: the guard strips literals and comments before matching, read both as standard SQL and as MySQL, because the two dialects disagree about what `\'` and `--` mean — a payload is flagged only if **either** reading still shows the keyword outside a literal. So `psql -c "SELECT 'DROP TABLE x' AS s"` and `psql -c 'SELECT 1 -- DROP TABLE x'` are both allowed (`plugin/crew/tests/test_cloud_guard.py`, cases `sql-literal/sql-comment/sql-block-comment`). If a query like this is still blocked, the reason (`[sqlDestructive]` in the deny message) is worth reading closely — the keyword may genuinely sit outside every literal reading.
- **A dry-run, help, or -whatIf invocation is still classified as destructive.** The guard matches on the *subcommand*, not on flags that make it a no-op: `terraform destroy --help`, `aws ec2 terminate-instances --dry-run` and `Remove-AzResourceGroup -WhatIf` are all judged exactly like the command they simulate, because nothing in `_terraform_destructive/_aws_destructive/the Remove-Az*` matcher inspects those flags. This is a known gap, not a config bug — there is no key that special-cases them. If you need to run one of these unattended, use `report` mode while testing, or approve the one-shot marker the deny message names. A literal `gh workflow run ... --help` (or `-h`) is the exception: `gh` prints help and dispatches nothing, so the line is not judged at all; `terraform's --help` still is.
- **A workflow dispatch is refused as "could not tell" although it looks fine.** With `environments.workflows` set, a `gh workflow run` or `gh api .../dispatches` line is judged only when every word on it is a plain literal (letters, digits, `_`, `.`, `:=@%+ , -`) or one whole single-quoted word, joined only by `;`, `&&`, `||`, `&`, newlines, `>/>>/&>/&>>` to a plain word and

2>&1. A pipe, any <, double quotes, \$, a glob, bash -c '...', xargs, a command word built at run time (\$X \$Y run ..., Start-Process \$x, an alias to a variable), gh reading stdin or a file (--json, --input, -F k=@f), and a malformed environments block in either config layer make the line could-not-tell: it asks when you are there and is denied unattended at every setting. **Fix:** write the literal form — -f environment=staging instead of a --json body, 'Deploy Staging' instead of "Deploy Staging", > log instead of | tee log, the value instead of \$ENV — or approve the one command with the marker the refusal names (it covers those exact bytes only). Other gh commands (gh pr create --title "...") are never gated.

- **An "unpinned" repo makes an ordinary destructive command ask or deny, even when it is obviously fine.** With no cloud.* pins, a read-only command (aws s3 ls, az group list) runs unchecked — but *any* destructive command has an identity the guard cannot vouch for, which is **unknown**, and unknown is never allowed unattended: it **asks** when a person is there to answer and is **denied** when not (CREW_UNATTENDED=1, CI, or a non-interactive session). This is the rule most likely to surprise a first-time user, because it fires even in a repo that has never configured anything cloud-related. **Fix — pin the identity this repo may act as**, in .crew/config.json (repo-only; a machine-global cloud block is ignored):

```
{ "cloud": {  
  
  "awsProfiles": ["acme-dev", "acme-sandbox-*"],  
  
  "awsRegions": ["eu-west-1"],  
  
  "azureSubscriptions": ["00000000-0000-0000-0000-000000000000", "Acme Dev"] } }
```

A profile/region/subscription outside the pins is denied outright ([cloudIdentity]). Static AWS_ACCESS_KEY_ID credentials and Remove-Az*'s saved PowerShell context are **always** unknown — the guard has no way to read either — so pinning does not silence those; only removing the static keys or naming the account through --profile/--subscription does.

Unattended Launch Refuses

crew_unattended.py launch -- claude ... starts an unattended session holding sealed, owner-named cloud credentials, or refuses and starts nothing (plugin README, "Unattended runs: sealed cloud credentials"). Run python3 "<crew>/hooks/scripts/crew_unattended.py" check --root . to see every check without launching. Each line is ready, refuse or unknown (could not tell,

which also refuses).

- **config: no read-only identity named** (or no `unattendedCloud` in the machine config). The defaults name nothing. **Fix:** name the read-only role in `~/.claude/crew/config.json`, never in the repo (a repo copy is ignored and reported): set `unattendedCloud.aws.readOnly.profile` and `unattendedCloud.aws.readOnly.identity`, the assumed-role ARN prefix ending in `/`.
- **config: environment X is not nonProd or no machine entry for nonProd environment X.** `--environment` needs both the repo's `environments.nonProd` to match the name and a machine `unattendedCloud.aws.nonProd` entry for it. There is no production entry, by design.
- **provider not implemented.** Only `aws` is read; remove the other key.
- **export: no SessionToken ...: these are static keys.** The profile resolves to long-lived keys. **Fix:** point it at an SSO or assume-role profile; crew never runs `aws sso login` for you.
- **export: credentials expire in under 15 minutes** or `export ... exited:` refresh the profile's session yourself (`aws sso login`), then retry.
- **identity: STS says ..., expected** The exported credentials are a different role than the one named; a `:user/` ARN is a long-lived IAM user and always refuses. Fix the profile, or the prefix if the role was renamed.
- **sandbox: unavailable: apply-seccomp: write /proc/self/setgroups** Claude Code's sandbox starts but cannot run commands on this host (measured with `kernel.apparmor_restrict_unprivileged_userns = 1`). Every launch refuses until the host owner makes the sandbox usable or runs unattended work as a separate OS user or container.
- **settings: <file> sets sandbox or allows Read(...).** The repo's `.claude/settings.json` or `.claude/settings.local.json` tries to shape the sandbox or widen reads. The sealed session never loads those files, but the launcher refuses rather than trust that. **Fix:** remove the key from the repo's file, or run that work attended.
- **settings: ... sandbox.excludedCommands, allowRead entry ... could re-open a credential store or sandbox.filesystem.disabled.** Your `~/.claude/settings.json` loads in the sealed session and would let a command run unsandboxed or re-open a store. **Fix:** move the entry out of your user settings (or narrow the `allowRead` away from the stores), then retry.

- **sandbox: the session can read a store: OPEN <path>.** The sealed settings did not hold for that path (a managed or user setting may be widening it). Nothing launches; find the setting.
- **sandbox: store not reported, could not tell, no end marker, or probe made no tool call.** The probe did not measure every store, or its output was cut short; retry, and treat a repeat as a host problem, not a pass.
- **refuse command:** launch runs claude only (an executable file or a name on PATH), and supplies --settings and --setting-sources itself.

Obsidian

See [Memory and Obsidian](#) for setup. What goes wrong day to day:

- **Symptom: nothing is being captured / gardened. Check:** is the *primary* vault reachable? Capture, import and the gardener write to the primary vault only, and **never** fall back to a recall vault. If the primary is unavailable — an unmounted drive, a renamed folder — they write nothing and say why on stderr; the session itself is never blocked. \$V0 adopt shows which vault is currently primary.
- **Symptom: recall never surfaces anything you know is in the vault. Check:**

```
python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats
```

Read the miss reasons line. cli-missing means obsidian-vault is not installed where crew can find it; cli-exit-N, cli-timeout, cli-bad-json mean the CLI ran and failed; no-vaults means no vault is currently primary or recall; no-hits means the search itself found nothing. Only no-hits means "the vault genuinely has nothing relevant" — every other reason is a setup problem. Also confirm memory.inject is not false in .crew/config.json — recall and code-map injection are on by default since 1.0.0, so "no recall ever appears" with the key set to false is expected. **Fix:** \$V0 recall --query "<words you expect>" directly, to separate "the CLI can't find it" from "the hook isn't calling the CLI". An empty answer can be the CLI's relevance floor or its excluded folders, not a missing note: the obsidian-vault recall CLI drops stop words, needs more than one matching word for a longer query, matches whole words only, and never reads wiki/sessions/archive/. Re-run with --min-terms 1 (the floor off) and --include-excluded (read the archive) and read below_floor and excluded_dirs in the --json output to see which one hid it.

- **Symptom: the gardener seems to be falling behind.** It runs in bounded passes on purpose, not continuously: **at most 5 items or 10 minutes per run**, whichever comes first, on **one designated host**, one run at a time. A session whose transcript lives on another machine is left queued for that machine and does not spend one of the five slots. **Check:** `$V0 queue` (what is waiting) and `~/ .claude/obsidian/gardener.log` for lines starting `left queued`. **Fix:** run a bounded pass by hand (`/obsidian-vault:garden` or `$V0 garden-run`), or drain a large backlog in batches: `$V0 drain --apply --batches 4`.
- **Symptom: two machines sharing one vault seem to be racing each other.** They are not writing the same file: each host appends to its own `inbox/pending-reflect.<host>.md` and acknowledges to its own `inbox/reflected.<host>.md`, so two machines syncing one vault never contend for the same queue file. An older `inbox/pending-reflect.md` (no host suffix) is still read if present, so an old backlog drains rather than being stranded — it is just never written again.
- **Symptom: `/crew:brainstorm` or `/crew:fix` stops because create refused a ticket id.** On the Obsidian ticket board, `crew_tracker.py create` refuses an id it cannot prove is yours, and prints one of four lines (paths and ids here are examples). The first three begin `id taken`, and `brainstorm` and `fix` then take the next free id on their own. For any other refusal they stop and show you the line.
 - **Another repo owns it.** `obsidian: could not update: id taken: T-0060` on `Board.md` belongs to another repo (`https://example.invalid/team/app`, per `T-0060.md`), not this one (`https://example.invalid/other/app`): give this repo its own `obsidian.boardDir` **Fix:** two repos share one board. Give each repo its own `obsidian.boardDir`.
 - **An older crew wrote the note's id lowercased.** `obsidian: could not update: T-0042` on `Board.md` belongs to another repo (`https://example.invalid/team/app`, per `T-0042.md`), not this one (`https://example.invalid/Team/App`): give this repo its own `obsidian.boardDir`; the note may carry this repo's id as an older crew wrote it (lowercased): if the card is this repo's, change the line to `'repo-id: https://example.invalid/Team/App'` **Fix:** since T-0071 an origin keeps the case of its user and path. If the card is yours, change the note's `repo-id:` line to the id it names.
 - **Nobody can tell who owns it.** `obsidian: could not update: id taken: could not tell whose card T-0042 is` (no `T-0042.md` note names its `repo-id`); if it is this repo's, put `'repo-id: https://example.invalid/other/app'` in `T-0042.md` **Fix:** if the card is yours, add the line it names to the note (see "The ticket

board" in [Memory and Obsidian](#)). If it is not yours, let brainstorm take the next id.

- **Another session in this repo holds it.** files: could not update: id taken: T-0050 is already ready "Fix tests" **Fix:** none needed; .work/INDEX.md already has that ticket. This check runs before the vault is opened, so it wins even when the vault is missing.
- **Crew cannot tell which repo this is.** obsidian: could not update: could not tell this repo's identity: git gave neither an origin URL nor a common dir for <repo path> **Fix:** run crew inside a git checkout. `git rev-parse --git-common-dir` must answer there.

Auto Wrap-Up, Clear and Resume

See [Auto wrap-up, clear and resume](#) for the full cycle that lets a long session finish itself cleanly, clear, and pick up where it stopped — what wrap-up, auto-clear and resume each do, the machine-global keys that turn auto-clear on, and every reason a clear can refuse (logged to `.crew/.autoclear.log`).

"Auto-Clear Did Nothing on Windows"

Symptom: auto-clear is on, a handoff is written and verified, and nothing types the configured command — no keystrokes appear at all.

- **Check `context.autoClear.method`** with `/crew:config --explain` or `crew_config.py --explain`. On native Windows with no tmux pane, "auto" (the default) resolves to "notify", not to any keystroke method — **this is not a bug**, it is the 1.0 owner decision: auto never arms SendKeys on your behalf. "notify" types nothing at all; instead the Stop hook prints a `systemMessage` telling you the handoff is written and verified and it is safe to run the configured command yourself.
- **If you want keystrokes typed for you**, ask for sendkeys by name — it is opt-in only, and `/crew:init`, `/crew:migrate` and `/crew:onboard` all refuse to write it without an explicit yes. It declines and falls back to notify (logged to `.crew/.autoclear.log`) inside Windows Terminal specifically, because that host puts every tab in one window and nothing outside it can confirm which tab is active — this is also not a bug, and switching terminal hosts is the only fix.
- **Fix:** either treat the notify message as the intended behaviour, or run `crew_autoclear_setup.py apply-method sendkeys --yes`

(`${CLAUDE_PLUGIN_ROOT}/hooks/scripts/`) to opt in, after reading what SendKeys does.

See `plugin/crew/CONFIG.md` §14.

"Bare `/crew:autopilot` Stopped on a Goal"

Symptom: `/crew:autopilot` with no argument stops naming autopilot goals, or a fell through: line names a stopped goal.

With no usable handoff, a bare run looks for a running goal in `.work/autopilot/` before the active ticket (the order is argument, handoff, running goal, active ticket, INDEX; L-0659).

- **several autopilot goals are running: a, b:** pick one: `/crew:autopilot --goal <slug>`.
- **could not tell whether an autopilot goal is running: `.work/autopilot/<slug>.json` ... could not be read:** the file is not JSON, not an object, or its `run.state` is not running, stopped or done. Fix or remove it; until then the active ticket is not driven, because a goal might be running. Naming the work (`/crew:autopilot <ticket>` or `--goal <slug>`) still runs.
- **goal `<slug>` stopped: `<reason>` as a fell through: line:** a goal you or a cap stopped is never resumed by a bare run; `/crew:autopilot --goal <slug>` resumes it when you choose to.

"Claude Code Compacted by Itself"

Symptom: the session cleared or summarised itself with no `/clear` or `/compact` typed, and no message about auto-clear ran first.

That is Claude Code's own **built-in auto-compact**, not this plugin. Crew neither causes nor tunes it — `context.autoClear` and `context-watch.sh` only ever act *after* a handoff is written and verified, and nothing in this plugin can trigger a compaction on its own. There is no crew setting that changes when Claude Code's auto-compact fires.

The one thing crew does around either kind of reset: on the next `SessionStart` after any `/clear` or `/compact` — self-initiated or Claude Code's own auto-compact — `handoff-read.sh` reloads `.work/HANDOFF.md` back into context automatically, so a compaction you did not ask for still resumes from the last written handoff rather than from nothing.

Autopilot Refused a Successor Plan After an Automatic Reject

- **Symptom:** `/crew:autopilot` stops with refused: successor plan lacks `<n>` of `<m>` finding line(s), first: `<line>` after it rejected a review round itself (`autopilot.maxAutoReplans`).
- **Cause:** after an automatic reject, autopilot approves a successor plan only when `plan.md` holds every BLOCK| and FIX| line of the rejected round verbatim, each as a whole line, as many times as the round carries it (L-0670). NIT lines are not required. could not tell: ... means the ledger, the rejected round's row or `plan.md` could not be read.
- **Check:** `python3 plugin/crew/hooks/scripts/crew_autopilot.py replan-check --root . --ticket <id>` prints applies= ok= missing= and the reason, and writes nothing.
- **Fix:** quote the missing lines in the plan, or approve it yourself with `/crew:approve <id>`, which this check never blocks.

An Agent Named in verify.json Is Not Installed

`.crew/verify.json` travels with the repo; the agents its rules name do not. A rule asking for an agent this machine lacks reviews less, and nothing in the output says so.

- **Symptom:** `/crew:status` prints agents MISSING `<name>` (`verify.json rule: <paths>`), or `/crew:review` reports a requested agent as a gap. **Check:**

```
python3 "<crew>/hooks/scripts/verify_agents.py" --root . --check
```

Exit 1 lists each missing name with its rule's paths. Exit 2 (unknown) means a plugin registry, a settings file or `verify.json` itself would not parse, so it could not tell; it never reads that as installed. Managed-policy agents and `--agents agents` are not checked. **Fix:** install the plugin or agent, enable the plugin (`enabledPlugins`; a narrower settings scope's `false` wins), or change the rule to an agent this machine has.

The Temp Directory Fills with Crew Files

Earlier crew releases' test suites and auto-clear sender could leave files in the system temp directory: `crew-completion-audit.*` markers, `tmp.*` auto-clear sender scripts (`tmux send-keys ... /xdotool ...` followed by `rm -f -- <itself>`) and `tmp.*` fixture directories from crew's

shell regression suite. Enough of them exhaust the inodes, and the guard and the Stop verify-gate then fail closed. Now every crew test runs with its own TMPDIR, the sender deletes itself before it sleeps, and the shell suite removes every fixture it makes.

- **Symptom:** `df -i /tmp` near 100%, or hooks failing on `mktemp`. **Check** (read-only):

```
cd "${TMPDIR:-/tmp}"

ls -d crew-completion-audit.* 2>/dev/null | wc -l

grep -l -e '^tmux send-keys -t' -e '^xdotool ' tmp.* 2>/dev/null | wc -l
```

Fix (an owner action; crew itself never runs it): with no crew session or test run active, remove what the check counted:

```
cd "${TMPDIR:-/tmp}"

find . -maxdepth 1 -name 'crew-completion-audit.*' -mmin +60 -delete

grep -l -e '^tmux send-keys -t' -e '^xdotool ' tmp.* 2>/dev/null | xargs -r rm -f --
```

Old shell-suite fixture directories are ordinary `tmp.*` directories (a `.crew/` and a `.git/` inside); read each before removing it.

A Setting Seems to Do Nothing

Symptom: you set a key in `.crew/config.json` or `~/.claude/crew/config.json` and crew behaves as if it were absent.

- **Check:** the Inert settings (crew <version> does not act on them): ... line at session start, the inert line in `/crew:status`, or `python3 "<crew>/hooks/scripts/crew_config.py" --root . --inert`. Each names key=value (why): a ticket id means the installed crew does not implement it yet; unknown key means a typo or a key from another crew version; global, repo-only means the machine file may not set it, so it takes effect nowhere.
- **Fix:** move a repo-only key into the repo's `.crew/config.json`, correct a typo, or wait for (or install) the crew version that brings the ticket. Nothing is refused while a key is inert.

Graph Refresh Refused: Secrets-Denylisted Path

graphify reads every file its ignore rules do not exclude and records the symbols it finds in `graph.json`. For a file git tracks, `.gitignore` does not stop it: only `.graphifyignore` does. So crew checks, before it names or runs a graph build, that the root `.graphifyignore` excludes every file the secrets denylist matches.

- **Symptom:** `refresh-check` prints `graph graphify-out: unknown - graphify would read N secrets-denylisted path(s) .graphifyignore does not exclude: ...; stop - a refresh cannot settle this`, or `/crew:status` prints `graph-ignore UNCOVERED`. **Check:** which files, and from which source.

```
python3 "<crew>/hooks/scripts/crew_graph_ignore.py" --root . --check --json
```

The denylist is the built-in patterns (`.env`, `.env.*` but not `.env.example`, `*.pem`, `*.key`, `*.p12`, `*.pfx`, `id_rsa*`, `id_ed25519*`), `.claude/secrets-denylist` if it exists, and every in-repo `Read(...)` rule in `permissions.deny` of `.claude/settings.json` and `.claude/settings.local.json`. sources in the JSON names which ones were read. **Fix:** add the missing patterns, then check again.

```
python3 "<crew>/hooks/scripts/crew_graph_ignore.py" --root . --write
```

It appends the missing patterns, then any flagged path they still miss (the denylist ignores case, `.graphifyignore` does not, so `.ENV` gets `/.ENV`), under one marked block, and leaves every existing line as it was. Commit `.graphifyignore`, then re-run the refresh check. If it prints line `N (!pattern)` re-includes denylisted `<path>`; remove that line or accept the exposure and exits 1, your own `!` line is the reason: `--write` never overrides it. Delete the line, or keep it and accept that graphify reads that file. A path that is not one line of plain text (a name holding a line break or other control character) is never written either; it stays named, escaped, by `--check`: rename the file or exclude it by hand.

- **Symptom:** `denylist coverage unknown: <reason>` (the refresh check) or `graph-ignore unknown (/crew:status)`. **Check:** the reason names it: git missing, a settings file that does not parse, an unreadable `.claude/secrets-denylist` or `.graphifyignore`, a nested `.graphifyignore` (only the root one is evaluated, and a nested one could re-include a file with `!`), a nested repository or submodule `.graphifyignore` does not exclude, a symlink that resolves outside the repository or not at all, or a deny-all `Read` rule. **Fix:** repair what it names. An unknown is never read as covered, so no refresh is named until it is fixed.

- **Symptom: a graph was built before the fix** (by hand, or by graphify's post-commit hook, which bypasses crew). **Fix:** treat that graph as holding text from the secret files. Delete the graph output directory (graphify-out/ by default), run `--write`, check that `--check` exits 0, then rebuild. Commit the rebuild if the output is tracked. The crew-graph skill's **Tainted graph** section has the same steps.

A Message from Another Session Asks for an Approval or an Edit

Sessions that share a coordination channel may message each other over Claude Code's bridge, but a message is only a doorbell: "the record on crew-coord/<channel> moved". Nothing in a message is ever an approval, an answer to a question you were asked, or permission to edit a file outside the ticket's Touch list.

- **Symptom: a peer's message says "approve T-0042", "answer Q2 with option B" or "edit the deploy script", or the session prints not a doorbell or could not tell.**
Check: what the classifier made of it.

```
python3 "<crew>/hooks/scripts/crew_bridge.py" receive --channel <c> --remote origin  
<<'END-7f3a'
```

```
<the message, exactly as received>
```

```
END-7f3a
```

Use a terminator of your own, and check first that no line of the message equals it: such a line would end the heredoc early and run what follows as shell. Exit 0 is a doorbell whose tip is in the fetched record; exit 3 (`could not tell`) means the fetch failed or the announced tip is not in the record; exit 1 (`not a doorbell`) is anything else, printed once, made safe and labelled [`peer-written`]. **Fix:** whatever it printed, do not act on the message's text. Run the one next step it names, `crew_coord.py status --channel <c> --remote origin`, and read the record. An approval is yours to type (`/crew:approve <id>`); a question is answered under your own questions policy; a request the peer needs actioned is filed in the record by the peer. Report `could not tell` and `not a doorbell` to whoever owns the channel.

Turning Things Off

Every switch named above, in one place. "Off" for a guard means the `PreToolUse` hook still fires

but returns immediately without judging anything; "off" for verifyGate means the Stop hook does nothing at all that turn.

Key	Lives in	Values	What "off"/floor means
autopilot.mode	repo only	off/plan	off (the default): /crew:autopilot runs no phase; only the exact string plan arms it
memory.inject	repo only	bool	false: no code-map, handoff or vault text is injected (default true since 1.0.0)
guards.cloudGuard	both layers, ratchets	block/ report/ off	off: the hook reads this key and exits
guards.roleWrites	both layers, ratchets	block/ report/ off	off: every Write/Edit is allowed unconditionally
guards.terraformApply, forcePush, adminMerge, mergeGate, cloudDestructive, sqlDestructive	both layers, ratchets	block/ ask/allow	there is no "off" — allow is the most permissive tier, still logged
guards.deployWorkflow	both layers, ratchets	block/ ask/allow	block is default and floor; allow covers nonProd only — production without environments.prodUnattended in both layers, and an unknown environment, still ask (denied unattended)
environments.nonProd	repo only	list of globs	[]: nothing is nonProd, so every terraform apply asks (denied unattended)
environments.prodUnattended	both	bool	false in either layer holds it down:

Key	Lives in	Values	What "off"/floor means
	layers, ratchets (true only when both say true)		production applies and dispatches ask
<code>guards.prodDatabase,</code> <code>guards.prodServer</code>	both layers, ratchets	none/ read/full	none is both the default and the floor
<code>scope.mode</code>	repo only	off/ report/ block/ auto	off: neither the edit guard nor the completion audit runs
<code>scope.allowCliApproval</code>	repo only	bool	false: only a /crew:approve typed by the user counts (no cli or autopilot receipt)
<code>verifyGate</code>	repo only	bool	false: the Stop verify gate does not run at all
<code>context.enabled</code>	both layers	bool	false: context-watch.sh (the Stop nag) does nothing
<code>context.autoClear.enabled,</code> <code>autoWrapUp</code>	both layers	bool	each independently disables one leg of the wrap-up/clear loop
<code>notify.provider</code>	both layers (a repo null inherits the global	telegram/ teams/ none	a repo none opts out even when the global file names a provider: no question ping, no deploy result (crew_notify.py config --root . says so)

Key	Lives in	Values	What "off"/floor means
	one)		
	both		
	layers,		
	ratchets		
change.requireForProduction	(may only turn ON)	bool	false: promoting needs no approved change request
		see	
install.policy	both layers, ratchets	plugin/crew/CONFIG.md \$"install"	narrowest tier refuses more install actions

A destroy is never applied unattended, at any setting: destroy, apply -destroy, -replace, workspace delete, a saved plan whose sidecar lists a delete, or an apply with no sidecar at all asks when you are there and is denied unattended — terraformApply: allow and environments.prodUnattended included. The full decision table is in plugin/crew/CONFIG.md (environments.*).

~/ .claude/crew/config.json is the machine-global layer; .crew/config.json is per-repo and wins where both speak. A **ratcheted** key (marked above) can only be *narrowed* by the repo relative to the machine-global value, never widened — a repo cloned from someone else cannot silently loosen a guard the machine owner set to block. /crew:config --show shows where each setting actually came from, and /crew:config with no argument (alias /crew:config-setup) sets either layer from a menu: it marks a repo value that widens a guard with !, and names a repo value the machine layer holds down. The same menu can delete a repo's .crew/config.json: it moves the file to a backup in one rename, compares it with what the preview read (a file that changed since is put back and nothing is deleted), and prints the command that restores it for sh, cmd and PowerShell.

Web Testing

Symptom: --select web-testing (or the ticked default row) fails, or Playwright runs but the

visual rule never passes.

- **Node too old.** `install-prerequisites.{sh,ps1}` checks for Node ≥ 20.19 (or = 22.12) itself and refuses rather than installing or upgrading Node for you — `node -v`, install a newer one, and re-run `--select web-testing`.
- **Browsers missing system dependencies.** `npx playwright install --with-deps chromium` is what the installer runs; a bare `npx playwright install` (no `--with-deps`) skips the OS packages Chromium needs and produces a browser that launches, then crashes on first navigation.
- **Docker absent -> visual UNVERIFIED, not a fail.** `toHaveScreenshot` baselines are only meaningful when generated inside `mcr.microsoft.com/playwright:v1.63.0-noble` — the installer warns rather than blocking when Docker is missing, and the `visual.verify.json` rule is written to skip (report UNVERIFIED) outside that image rather than compare against a baseline that can never match.
- **MCP server won't connect on Windows.** Both `@playwright/mcp` and `chrome-devtools-mcp` are launched with `npx`, which the MCP host cannot invoke directly on Windows — wrap the entry as `"command": "cmd", "args": ["/c", "npx", ...]` in `.mcp.json` (documented in `chrome-devtools-mcp`'s own troubleshooting for the "Connection closed" failure); the `stack-web` skill's `.mcp.json` example already shows this.

Windows Notes

- **pwsh is not on Git Bash's PATH.** Every script that shells out to PowerShell names it by an absolute path or resolves it explicitly; a bare `pwsh` in a hand-typed command fails as "command not found", which is easy to misread as PowerShell itself being broken.
- **Git Bash ships without python3.** Every hook wrapper here resolves `python3`, then `python`, then `py`, and fails loudly on `stderr` rather than silently standing down when none resolves (see "no-python fail-closed", above). If a hook you expect to run says nothing at all, check for a usable Python on PATH first.
- **Both flavours are always registered.** `hooks.json` pairs every bash command with a `shell: "powershell"` sibling on the same event, so a machine with both bash and pwsh

present runs both. This is not double-firing to fix — it is how the same hook reaches both shells; each PowerShell twin stands itself down on the wrong platform (`$env:OS -ne 'Windows_NT'`) rather than the bash twin doing that job.

- **Tests and checks are slow on Windows.** Read `/crew:status`'s shell line first. It names the shell route crew's jobs take (`shellRoute.mode`: `auto`, `wsl`, `powershell`, `gitbash`) and why. `WSL never probed` means nobody ran `crew_shell.py probe --write`; `not-installed` comes with the `wsl --install -d Ubuntu` recommendation, which crew prints and never runs. A repo on a Windows drive goes to WSL under `auto` only after `crew_shell.py measure --write` has shown WSL faster there. See "Choosing the shell route on Windows" in `plugin/crew/skills/crew-setup/platform.md`.

Auto Wrap-Up, Auto-Clear and Auto-Resume

This section is folded into the troubleshooting guide (`troubleshooting.md`), linked from it as "Auto wrap-up, clear and resume". It covers the cycle that lets a long session finish itself cleanly, clear, and pick up where it stopped.

What the Cycle Does

1. **Wrap-up.** When the context window crosses its threshold, the Stop hook blocks once. It asks the session to finish or park the change in flight, write `.work/HANDOFF.md`, and update the ticket. This happens once for each threshold crossing in each session. It never happens on the forced continuation that follows a block. With `context.autoClear.wrapUp` armed (below), the block carries the wrap-up procedure instead, and the clear waits for its results.
2. **Clear, or notify.** After the handoff is written, auto-clear acts on it. What it does depends on method (below): `notify` (the default on native Windows) prints a message telling you the handoff is written and it is safe to run `/clear` (or `/compact`) yourself — it types nothing. `tmux`, `xdotool` and `sendkeys` actually type the configured command into the terminal that runs this session. Whichever method is in force, it acts only when all five of these hold: - this machine opted in; - the handoff was written by this session after the wrap-up request, is longer than a stub, and is not the automatic `PreCompact` skeleton; - the context reading that caused the wrap-up was a real measurement (see "When it will not clear"); - this session is bound to its own Claude Code process, and that process has a terminal of its own (see "Which window gets the keystroke") — a headless session, such

as a `claude -p child`, gets one message instead, naming its handoff and saying the process that started it must start a new one; - for a method that types (`tmux/xdotool/sendkeys`), the target window can be identified exactly, from that process — `notify` needs none of this, since it identifies no window.

Crew neither causes nor tunes Claude Code's own auto-compact. That is a separate, built-in behaviour that fires whenever Claude Code itself decides to; this cycle only reacts to a handoff once one exists, and nothing here makes auto-compact happen sooner, later, or at all. While an autopilot goal runs, the handoff names the goal, not the ticket in hand: every writer (autopilot's low-context stop, `/crew:handoff`, the PreCompact skeleton) asks `crew_autopilot.py handoff-resume`, which answers `resume: /crew:autopilot --goal <slug>` for the one running goal. Two running goals or an unreadable goal file give `resume: none`: it could not tell which goal runs, and it never falls back to the ticket form, which would drop the goal. With no usable handoff at all (a crash, a killed terminal), a bare `/crew:autopilot` looks for a running goal before the active ticket: the order is argument, handoff, running goal, active ticket, INDEX. One running goal resumes; two stop and list both; a goal file it cannot read stops (could not tell, so the active ticket is not driven); a stopped goal is named, never resumed. 3. **Resume.** The new session starts with the handoff and its next action already in context, within the 3,000-character resume budget. Nothing starts working unprompted: crew does not use `initialUserMessage`, so you press Enter to continue. Claude Code 2.1.282 drops it in an interactive session anyway (spike, 2026-09-25): it is honoured by `claude -p` only.

With `resume.auto: true` in the machine-global config, the handoff after `/clear` or a manual `/compact` also names the exact next command, read from the note's `resume: line` (`resume: /crew:done T-0001`): Auto-resume: ready to run `/crew:done T-0001`. — or Auto-resume did not start: `<reason>`. when a check refuses it. It does not start from the hook: you press Enter or type it, or crew types it where the terminal can be identified (below). The refusal reasons are: compact was not a manual `/compact`; no handoff note, or it was archived as stale, or is stale and could not be archived; the handoff is the automatic PreCompact skeleton; no resume line, `resume: none`, or a line the grammar refuses (a second line, trailing text, an unknown command, or an excluded one such as `/crew:approve`); the note's `branch: or head:` does not match the checkout (a ticket line only: a `--goal` line is judged by its goal file, and waits unless that goal is running); the ticket directory or goal file is missing; the command is not installed; `handoff-author.json` could not be read; no record of which session wrote this handoff; a later handoff write could not replace or remove `handoff-author.json` (`handoff-author.json.stuck`), or the file and its directory are both read-only so it can be neither replaced nor removed; the handoff changed since its author session wrote it; the handoff was written by another session; this

session's process could not be identified (always on a host without `/proc`, such as native Windows or macOS); the record of past auto-resumes (`resume-state.json`) could not be read, or its directory cannot be searched; this handoff was already resumed; the progress fingerprint could not be computed; the same command with no progress since the last auto-resume; `internal error` (the decision itself failed, and the handoff is still injected). Never on a plain startup.

Crew types it where it can (T-0013). The context hook runs its own flavour's `auto-clear` sender in resume mode. Consent is `resume.auto`; `context.autoClear` only describes the terminal. In `tmux` it waits `resume.typeDelaySeconds` (default 2), polls the pane for an idle, empty input line for up to `resume.readyTimeoutSeconds` (default 15), then sends the command and, half a second later, `Enter`. On native Windows it types only under an explicit method: `sendkeys`, after the delay, with the focus and tab rechecks; `auto` there types nothing. It types once per handoff (a marker under `<git-common-dir>/crew/`) and records the run first, so a run that could not be recorded is never typed. The context then says `Auto-resume: typing /crew:done T-0001 into this session in 2s (method tmux)`, or keeps the line above and adds `Auto-resume was not typed: <reason>`. Every refusal is in `.crew/.autoclear.log`: `decide did not say run; no usable method, $TMUX unset` or a pane that is not this session's; `wtype` or `xdotool`; already typed; the run could not be recorded; the input line is not empty; not ready in time; on Windows, focus lost or a tab not provable. The 2 s default comes from a measurement (input ready by 0.134 s after `SessionStart` on Claude Code 2.1.282); a delay is still a guess, and Windows has no probe.

A note resumes only in the session that wrote it. Write it with the `Write` tool (as `/crew:handoff` does): the context hook then records which session and which Claude Code process wrote it. After `/compact` the session must match; after `/clear` the process must, so a `/clear` in another terminal on the same worktree waits. A note written by `Bash` or by hand waits.

Turning It on and Off

Wrap-up and resume are on by default. Auto-clear is **off** by default. You turn it on per machine, in the machine-global config, because it drives this machine's keyboard:

Where	Key	Value	Effect
<code>~/.claude/crew/</code> <code>config.json</code> (Windows: <code>%USERPROFILE%</code>	<code>context.autoClear.enabled</code>	<code>true</code>	turns auto-clear on for every crew repo on this machine

Where	Key	Value	Effect	
\.claude\crew\ config.json)				
	same	context.autoClear.method	"auto" (default), "tmux", "xdotool", "notify", "sendkeys", "none"	how the cycle acts — see "Which method, and what it does" below
	same	context.autoClear.windowTitle	a substring of the terminal's title	fallback when the window cannot be found by process id
	same	context.autoClear.delaySeconds	3	wait before typing, so the prompt exists
same	context.autoClear.minHandoffLines	5	a shorter handoff counts as a stub	
.crew/ config.json (the repo)	context.autoClear.enabled	false	switches auto-clear off in this repo. A repo can switch it off, but true here does not switch it on.	
~/.claude/ crew/ config.json	resume.auto	true	names the next command from the handoff's resume: line after /clear or a manual /compact (off by default)	
.crew/ crew.json or	resume.auto	false	vetoed auto-resume in this repo. true here	

Where	Key	Value	Effect
.crew/config.json (the repo)			does not switch it on.
~/ .claude/crew/config.json	context.autoClear.wrapUp	true	turns the wrap-up into the wrap-up procedure and makes the clear wait for its results (off by default; acts only where enabled is armed). A repo false vetoes it; a repo true does nothing.
.crew/config.json	context.autoWrapUp	false	replaces the wrap-up instruction with a plain handoff request. It still blocks once. Armed wrapUp supersedes both wordings.
.crew/config.json	context.enabled	false	turns off the whole context watcher, wrap-up included
.crew/config.json	memory.inject	false	handoff - read prints the handoff for you to read, without extracting the next action.
			context.autoResume is no longer read.

/crew:config --show shows where each value comes from. /crew:config with no argument (or /crew:config-setup) opens a menu that sets either file from a list of values, with a dry run before anything is written. To turn the cycle off on a machine, set context.autoClear.enabled to false or delete the key. In a repo the menu offers only the veto (false) or null for

`context.autoClear.enabled` and `resume.auto`, and shows only `repos/onlySessions` read-only, because only the machine file can arm or narrow them.

Armed wrap-up (`context.autoClear.wrapUp`)

A hook cannot make the session do anything; it can only refuse to clear. So with `wrapUp` armed the threshold block sends one procedure — start no new step; run the step's `Test:` command (the active ticket's plan step, or the checks for the tracked diff with no ticket); commit only if it passes; if it cannot pass, leave the tree and write `resume: none` with the reason under **Verify first**; run `/crew:handoff --wrap-up`; end the turn — and auto-clear acts only when, beside the checks above:

- the handoff's head: is HEAD, so it was written after the commit;
- its branch: is the checked-out branch;
- no tracked file is modified (untracked files, and the handoff itself, do not count);
- its resume: line parses, or is `resume: none`.

Anything crew cannot tell — git failing, no `crew_resume`, on native Windows no python — refuses. A refusal is logged, shown to you as `crew wrap-up: not clearing - <reason>`, and handed back to the session once, at its next ordinary Stop. Crew checks that the commit happened, not that the test passed: the verify gate stands down on the turn the commit is made on.

The chain end to end: wrap-up (T-0017) → the target proven from this session's own process (T-0016) → clear → `resume.auto` decides (T-0006) → the command is typed (T-0013).

Arming one scratch repo while other sessions are live

`enabled: true` arms auto-clear in **every** Claude session on the machine, not just the one you are testing. If other sessions are running, one of them can get `/clear` typed into its terminal when its own handoff lands. To limit auto-clear to one repo or one session, add these keys to the same machine file. They can only narrow what enabled turns on:

Key	Value	Effect
<code>context.autoClear.onlyRepos</code>	list of absolute repo paths	auto-clear runs only in these repos
<code>context.autoClear.onlySessions</code>	list of session ids	auto-clear runs only in these

Key	Value	Effect
-----	-------	--------

sessions

- A missing key or `null` does not narrow anything. An empty list `[]` turns auto-clear off everywhere. If a key holds anything other than a list, auto-clear is off everywhere too.
- When both keys are set, a session must match both.
- Paths are compared after symlinks are resolved. Backslashes and trailing separators do not matter. On Windows, case does not matter and `/c/repos/x` means `C:\repos\x`. A relative path such as `.` never matches.
- Session ids must match exactly, including case.
- Finding the repos to list: pass `--scan-root <dir>` on the one `apply-migrate` run that `/crew:migrate` makes (`crew_autoclear_setup.py --root . apply-migrate --scan-root <dir>`); a later run cannot see this repo's own opt-in, which the first run removed. It looks under `<dir>` (3 levels by default, `--scan-depth <n>`), read-only, for repos whose `.crew/config.json` or `.crew/crew.json` still says `context.autoClear.enabled: true`, and proposes them with this repo. A repo it cannot read is listed and blocks `--yes-widen`. Repos outside `<dir>`, and repos an earlier migration already converted (their opt-in is gone), cannot be found: add those by hand.
- These keys are read **only** from the machine file. If a repo's `.crew/config.json` sets them, they are ignored, so a repo cannot add itself. A repo's `enabled: false` still turns auto-clear off.
- In any other repo or session, auto-clear stays silent. It does not write a log line, just as it does when `enabled` is off.

To arm one scratch repo safely:

```
{
  "context": {
    "autoClear": {
      "enabled": true,
      "onlyRepos": ["C:\\repos\\autoclear-scratch"]
    }
  }
}
```

```
}  
  
}
```

On Linux, use a path such as `"/home/you/autoclear-scratch"`. When you are done testing, remove `enabled` or set it to `false`. Also delete `onlyRepos`, so that a later `enabled: true` does not arm only that repo without anyone noticing.

Which Method, and What It Does

Method	What it does	Where auto picks it
<code>tmux</code>	Types the command into the <code>\$TMUX_PANE</code> whose process is this session's own process or an ancestor of it. No focus involved — the most reliable method.	Linux, macOS, WSL, Git Bash on Windows, when <code>\$TMUX</code> names a pane and <code>tmux</code> is on <code>PATH</code>
<code>xdotool</code>	Types into the one X11 window owned by the nearest ancestor of this session's own process, when that window hosts no other terminal (or matching <code>windowTitle</code>).	Linux with <code>xdotool</code> on <code>PATH</code> and <code>\$DISPLAY</code> set
<code>notify</code>	Types nothing. Prints a message saying the handoff is written and verified and it is safe to run the command yourself. Never says anything was cleared or compacted — it does not know that, and it did not do it.	native Windows , whenever there is no <code>tmux</code> pane — this is the default there, not a fallback for something missing
<code>sendkeys</code>	<code>System.Windows.Forms.SendKeys</code> against the one window owned by an ancestor of this session's own process (or matching <code>windowTitle</code>), confirmed still in the foreground right before typing. auto never picks this — typing into a window this hook	never (opt-in only)

Method**What it does****Where auto
picks it**

found itself is a risk auto does not get to accept on your behalf.

Set method: "sendkeys" explicitly to opt in. Even then, it declines and falls back to notify — logging why — when the target window is owned by Windows Terminal, because that host puts every tab in one window and nothing outside it can confirm which tab is active.

none

Refused outright.

never

When It Will Not Clear (or Notify)

Each refusal is logged with its reason to `.crew/.autoclear.log`. A Stop hook's stderr is not shown to you, so check that log first. A notify or sendkeys decline is logged the same way — "declined" rather than "refusing", since the machine has opted in and something still happened (a message, or a fallback), just not a keystroke.

Log says**Meaning**

not trustworthy (estimated-from-transcript-size)

no usage record was found, so the reading was an estimate

not trustworthy (unknown-window)

the model's context window is unknown and `context.budgetTokens` is not set

not trustworthy (stale-reading-before-compaction)

the last reading came from before a compaction

the wrap-up marker is empty or not JSON

the marker was not written by this version of the hook. Nothing clears on it.

predates this session's wrap-up request

the handoff on disk is older than the request, probably left by an earlier session

automatic PreCompact skeleton

only the compaction skeleton exists, not a real handoff

N windows have a title containing ... /

the window is ambiguous, so nothing is typed

Log says

has N windows

no window belongs to any ancestor of
this hook / ... of this session's process

sent - method notify-headless: crew:
this session has no terminal of its own
(...)

could not identify this session's
process (...)

could not tell whether this session has
a terminal of its own (entrypoint
'...' ...)

passes through another Claude Code
session (pid N)

the window's owner (pid N) also hosts
another terminal / another Claude Code
session

could not be listed / the parent of pid N
could not be read / deeper than 16
processes

found by its title alone, and another
Claude Code session is live / has no
owning process

tmux pane %N could not be confirmed

Meaning

(sendkeys/xdotool only)

the terminal's window could not be found by
process id, and no windowTitle is set
(sendkeys/xdotool only)

a headless session (claude -p: entrypoint sdk-
cli, or no controlling terminal): nothing was
typed, and the message names the handoff and
its resume: line. The process that started the
session restarts it

no Claude Code session record names a process
above the hook with this session's id and start
time (or the record is unreadable, or names
another session). Nothing is typed; auto falls
back to notify

the session record's entrypoint is not one
measured to have a terminal (only cli is).
Nothing is typed; auto falls back to notify

this session is a child of another session, and
the pane or window above it is the parent's

the terminal that owns the window also runs
another tab or session, so the window is not
provably this one's

the process tree could not be read far enough
to prove anything

a windowTitle match (or a window with no
process) cannot tell two sessions apart

\$TMUX_PANE is not the pane that runs this

Log says

Meaning

method wtype ... is refused

session

Wayland's wtype cannot target a window. Use tmux instead.

sent - method notify

the message was printed — this is the normal, successful outcome for notify

declined sendkeys - cannot verify the active tab ...

the target window is owned by Windows Terminal; sendkeys fell back to notify instead of guessing which tab is active

declined sendkeys - the target window lost focus during the ...s delay

the window had focus when identified, but not when the delay ended, so nothing was typed

Two sessions in one repository no longer interfere with each other. Wrap-up markers are `.crew/.handoff-requested-<session_id>`, and a `SessionStart` clears only its own session's marker.

Which Window Gets the Keystroke

Applies only to the methods that type — `tmux`, `xdotool`, `sendkeys`. `notify` identifies no window and needs none of this.

First, the session is bound to its own process: the nearest ancestor of the hook named by a Claude Code session record (`${CLAUDE_CONFIG_DIR:-~/ .claude}/sessions/<pid>.json`) carrying this session's id and that process's start time. Only a session whose record says `kind interactive` and `entrypoint cli`, with a controlling terminal (native Windows has none to read), is typed into. Every rule below walks up from that process, not from the hook, and refuses when the walk passes through another Claude Code session — a child started from a session's Bash tool inherits `$TMUX`, and without this its parent's pane is the one that got `/clear`. `CONFIG.md §14` has the full rules and their limits (Windows' entrypoint is unmeasured; macOS cannot check the start time).

- **tmux (Linux, macOS, WSL):** the pane in `$TMUX_PANE`, and only when that pane's process is this session's own process or an ancestor of it. No title is needed. This is the most reliable method.

- **X11 (xdotool):** the one visible window owned by the nearest ancestor of the session's process, and only when no other terminal (another tab's tty, another session) runs under that window's owner. If that process owns more than one window, `windowTitle` has to narrow the choice to one. If no ancestor owns a window, exactly one window must match `windowTitle`, and no other session may be live, since a title cannot tell two apart. The keystroke goes to that window id after it is activated and confirmed active.
- **Windows (sendkeys):** chosen the same way, by owner process first and then by title. Declines outright, before any of that matters, when the resolved window is owned by Windows Terminal — see the method table above. Otherwise, after the delay, the command is sent only when that exact window handle still has focus; if it does not, nothing is typed and the decline is logged.

Windows Terminal runs every tab in one window. A process-id match cannot tell which tab is active, so on Windows Terminal set `windowTitle` to something only this session's tab shows.